



Resumo estratégico A2

Desenvolvimento Back-end - Experiência Prática 2



2 Identificação do aluno

W(

**Walisson Francisco Barbosa Carvalho CRUZ_EAD_Ciência da Computação
(Bacharelado)_3A_20262**



Inicialização do ambiente

Criação e navegação para o diretório do projeto: `mkdir api-connect cd api-connect` Criação do ambiente virtual isolado (Virtual Environment): `python -m venv venv` O ambiente virtual (venv) isola as dependências do projeto, evitando conflitos de versões de bibliotecas globais na máquina do desenvolvedor. Ativação do ambiente virtual: No Windows (Prompt de Comando / PowerShell): `venv\Scripts\activate` No Linux / macOS: `source venv/bin/activate` Instalação do framework principal (Flask): `pip install flask` Instalação de ferramentas auxiliares de desenvolvimento e congelamento de dependências: Instalação opcional de utilitários como o `python-dotenv` para gerenciamento seguro de variáveis de ambiente: `pip install python-dotenv` Geração do arquivo de controle de pacotes do ecossistema Python (`requirements.txt`), garantindo a reprodutibilidade do ambiente em produção: `pip freeze > requirements.txt`



Código do servidor

```
from flask import Flask, jsonify, request # 1. Instanciação da aplicação Flask app = Flask(__name__) # Configurações do servidor PORT = 5000 HOST = '0.0.0.0' # 2. Middleware global (Interpretação e validação do formato JSON) @app.before_request def validate_json_content_type(): """ Middleware que atua antes de cada requisição. Garante que métodos que transportam payload (POST, PUT, PATCH) estejam operando estritamente sob o formato application/json. """ if request.method in ['POST', 'PUT', 'PATCH']: if request.content_length and request.content_length > 0 and not request.is_json: return jsonify({ "status": 400, "error": "Bad Request", "message": "O cabeçalho Content-Type deve ser 'application/json'." }), 400 # 3. Rota de verificação de integridade (Health Check) @app.route('/', methods=['GET']) def health_check(): return jsonify({ "status": "online", "servico": "API Connect - Gerenciamento de Usuários", "versao": "1.0.0" }), 200 # 4. Tratadores globais de erros de rede e formato @app.errorhandler(400) def bad_request(error): return jsonify({ "status": 400, "error": "Bad Request", "message": "Payload JSON malformado ou inválido." }), 400 @app.errorhandler(404) def not_found(error): return jsonify({ "status": 404, "error": "Not Found", "message": "O recurso solicitado não foi encontrado no servidor." }), 404 # 5. Inicialização do servidor em modo de escuta if __name__ == '__main__': print(f"Servidor da API Connect executando em http://localhost:{PORT}") app.run(host=HOST, port=PORT, debug=True)
```



Nome do arquivo ou diretório

- run.py (ou server.py)
- app/__init__.py
- app/routes/user_routes.py
- app/controllers/user_controller.py
- app/utils/validators.py
- app/data/users.json

Responsabilidade técnica

- Ponto de entrada (entry point) da aplicação. É responsável unicamente por importar a instância configurada do Flask, definir variáveis de ambiente de execução (como porta e modo de debug) e colocar o servidor em modo de escuta (listen/run). Não contém regras de negócio nem definição de rotas.
- Fábrica da aplicação (Application Factory). Configura a instância do Flask, registra os middlewares globais (como validação de cabeçalhos e parsing do formato JSON), manipula erros globais HTTP (como 400, 404 e 500) e acopla os Blueprints de rotas da aplicação.
- Camada de Roteamento (HTTP Routing). Define os endpoints da API (ex.: /users, /users/) e os métodos HTTP permitidos (GET, POST, PUT, DELETE) utilizando o recurso de Blueprints do Flask. Sua função restringe-se a receber a requisição da rede e delegar a execução imediatamente para a função co
- Camada de Controle e Lógica de Negócio (Controllers/Services). Extrai os parâmetros e o corpo (body) das requisições, orquestra as chamadas de validação de dados, executa as operações de CRUD sobre a camada de persistência e monta a resposta padronizada com o payload em JSON e o respectivo código de
- Camada de Validação e Sanitização. Isola as regras de integridade dos dados de entrada enviados pelo cliente nas operações de criação e atualização. Checa a presença de campos obrigatórios (como name e email), valida tipos de dados e garante que valores nulos ou vazios sejam rejeitados antes do proc
- Camada de Persistência Provisória. Armazena a coleção de registros de usuários em formato JSON estruturado (ou alternativamente gerenciado via lista de dicionários em memória). Garante que a aplicação possua um estado de dados desacoplado do código-fonte, simulando as operações de um banco de dados



Estrutura de dados inicial

Com base na árvore de diretórios estabelecida na etapa anterior (app/data/users.json gerenciado por uma camada controlada de persistência), foi implementado o módulo app/data/user_repository.py. Optou-se pela estratégia de arquivo JSON estruturado com fallback automático e sincronização em memória, garantindo persistência entre reinicializações do servidor. Para a identificação dos registros, utilizou-se uma técnica de IDs numéricos autoincrementais, simulando a chave primária sequencial de um banco relacional:

```
import json
import os

# Caminho absoluto para o arquivo de persistência JSON
DATA_DIR = os.path.dirname(os.path.abspath(__file__))
FILE_PATH = os.path.join(DATA_DIR, 'users.json')

class UserRepository:
    """ Camada de persistência provisória de dados para o MVP. Realiza a leitura e escrita síncrona em arquivo JSON local. """
    def __init__(self, file_path=FILE_PATH):
        self.file_path = file_path
        self._ensure_file_exists()
    def _ensure_file_exists(self):
        """ Garante a existência do arquivo JSON inicializado com dados mock. """
        if not os.path.exists(self.file_path):
            initial_data = [
                {"id": 1, "name": "Walisson Silva", "email": "walisson@email.com"},
                {"id": 2, "name": "Maria Oliveira", "email": "maria.oliveira@email.com"}
            ]
            self._save_to_file(initial_data)
    def _load_from_file(self):
        """ Carrega e desserializa os dados do arquivo JSON. """
        try:
            with open(self.file_path, 'r', encoding='utf-8') as f:
                return json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            return []
    def _save_to_file(self, data):
        """ Serializa e persiste a lista de usuários no arquivo JSON. """
        with open(self.file_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=4, ensure_ascii=False)
    def _generate_next_id(self, users):
        """ Técnica de ID incremental seguro: Identifica o maior ID presente na coleção e incrementa em 1. Caso a lista esteja vazia, inicia o índice a partir de 1. """
        if not users:
            return 1
        return max(user['id'] for user in users) + 1

# Operações da persistência
def get_all(self):
    return self._load_from_file()
def get_by_id(self, user_id):
    users = self._load_from_file()
    return next((user for user in users if user['id'] == user_id), None)
def create(self, user_data):
    users = self._load_from_file()
    new_id = self._generate_next_id(users)
    new_user = {
        "id": new_id,
        "name": user_data["name"].strip(),
        "email": user_data["email"].strip().lower()
    }
    users.append(new_user)
    self._save_to_file(users)
    return new_user
def update(self, user_id, update_data):
    users = self._load_from_file()
    for idx, user in enumerate(users):
        if user['id'] == user_id:
            if 'name' in update_data:
                user['name'] = update_data['name'].strip()
            if 'email' in update_data:
                user['email'] = update_data['email'].strip().lower()
            users[idx] = user
    self._save_to_file(users)
    return user
def delete(self, user_id):
    users = self._load_from_file()
    initial_length = len(users)
    users = [user for user in users if user['id'] != user_id]
    if len(users) < initial_length:
        self._save_to_file(users)
    return True
return False

# Instância global do repositório para injeção nos controladores
user_repo = UserRepository()

Justificativa Técnica da Implementação
Geração de ID Incremental (_generate_next_id): O método utiliza a expressão geradora max(user['id'] for user in users) + 1. Isso impede colisões de identificadores mesmo após operações de deleção intermediárias (diferente de usar len(users) + 1, que repetiria IDs caso o último elemento fosse removido). Isolamento de I/O: A manipulação direta de arquivos em disco fica encapsulada dentro da classe UserRepository, permitindo que os controladores manipulem apenas objetos de dados nativos (listas e dicionários do Python) sem conhecer a fonte de persistência subjacente.
```



Rotas POST e GET geral

```
from flask import jsonify, request from app.data.user_repository import user_repo def list_users(): """ Controlador para listagem geral de usuários. Método HTTP: GET /users
Retorno: Lista em JSON com status 200 (OK). """ users = user_repo.get_all() return jsonify({ "success": True, "total": len(users), "data": users }), 200 def create_user(): """
Controlador para criação de novo usuário. Método HTTP: POST /users Retorno: Objeto criado com ID gerado e status 201 (Created). """ # Extrai o corpo da requisição em
formato JSON payload = request.get_json(silent=True) # Validação sintática e de campos obrigatórios if not payload: return jsonify({ "success": False, "error": "Bad Request",
"message": "O corpo da requisição deve ser um objeto JSON válido." }), 400 name = payload.get("name") email = payload.get("email") if not name or not isinstance(name, str) or
not name.strip(): return jsonify({ "success": False, "error": "Bad Request", "message": "O campo 'name' é obrigatório e deve conter texto válido." }), 400 if not email or not
isinstance(email, str) or not email.strip(): return jsonify({ "success": False, "error": "Bad Request", "message": "O campo 'email' é obrigatório e deve conter texto válido." }), 400 #
Persiste o usuário utilizando o repositório com ID incremental new_user = user_repo.create({ "name": name, "email": email }) # Resposta obrigatória com status HTTP 201
(Created) return jsonify({ "success": True, "message": "Usuário cadastrado com sucesso.", "data": new_user }), 201
```




Rota GET por ID

1. Camada de Controle (app/controllers/user_controller.py) A função `get_user_by_id` extrai o parâmetro numérico, consulta a camada de persistência e avalia a existência do registro, retornando 200 OK em caso de sucesso ou 404 Not Found caso o ID não conste na base:

```
from flask import jsonify
from app.data.user_repository import user_repo

def get_user_by_id(user_id):
    """ Controlador para busca de usuário individual por ID.
    Método HTTP: GET /users/<int:user_id>
    Cenários de resposta:
    - Sucesso: Retorna o objeto do usuário com status 200 (OK).
    - Inexistente: Retorna mensagem diagnóstica em JSON com status 404 (Not Found).
    """
    # Consulta a camada de persistência
    user = user_repo.get_by_id(user_id)
    # Tratamento explícito para identificador inexistente
    if not user:
        return jsonify({
            "success": False,
            "error": "Not Found",
            "message": f"Usuário com ID {user_id} não foi encontrado na base de dados."
        }), 404
    # Retorno estruturado em caso de sucesso
    return jsonify({
        "success": True,
        "data": user
    }), 200
```

2. Camada de Roteamento (app/routes/user_routes.py) O endpoint recebe o path parameter tipado como inteiro (`<int:user_id>`), o que garante que requisições com strings não numéricas sejam automaticamente descartadas pelo próprio roteador do Flask:

```
from flask import Blueprint
from app.controllers.user_controller import get_user_by_id

# Adição da rota parametrizada ao Blueprint 'user_bp' (prefixo /users)
user_bp = Blueprint('user_bp', __name__, url_prefix='/users')
user_bp.route('/<int:user_id>', methods=['GET'])
def get_single_user(user_id):
    """ Rota parametrizada REST: GET /users/:id
    Delega a execução para o controlador get_user_by_id.
    """
    return get_user_by_id(user_id)
```

Detalhes Técnicos de Resiliência Prevenção de Colapso (Fail-Safe): A ausência do registro é tratada com uma condicional explícita (`if not user:`), evitando exceções em tempo de execução (`IndexError` ou `KeyError`) e garantindo que o cliente receba um corpo JSON padronizado. Semântica HTTP 404: O cabeçalho de resposta é despachado com código 404 Not Found, informando a clientes web e navegadores que o recurso identificado por aquele URI específico não existe no servidor.

Rotas PUT e DELETE

```
from flask import jsonify, request, Response
from app.data.user_repository import user_repo

def update_user(user_id):
    """ Controlador para atualização de dados do usuário por ID.
    Método HTTP: PUT /users/<int:user_id>
    Cenários de retorno:
    - 200 OK: Usuário localizado e atualizado com sucesso.
    - 400 Bad Request: Payload JSON ausente ou inválido.
    - 404 Not Found: Usuário com o ID especificado não encontrado.
    """
    # 1. Verifica a existência prévia do registro
    existing_user = user_repo.get_by_id(user_id)
    if not existing_user:
        return jsonify({
            "success": False,
            "error": "Not Found",
            "message": f"Não foi possível atualizar: usuário com ID {user_id} não existe."
        }), 404

    # 2. Extrai e valida o corpo da requisição
    payload = request.get_json(silent=True)
    if not payload:
        return jsonify({
            "success": False,
            "error": "Bad Request",
            "message": "O corpo da requisição deve ser um objeto JSON válido."
        }), 400

    # 3. Validação dos dados a serem atualizados
    name = payload.get("name")
    email = payload.get("email")
    update_fields = {}

    if name is not None:
        if not isinstance(name, str) or not name.strip():
            return jsonify({
                "success": False,
                "error": "Bad Request",
                "message": "O campo 'name' não pode ser vazio."
            }), 400
        update_fields["name"] = name.strip()

    if email is not None:
        if not isinstance(email, str) or not email.strip():
            return jsonify({
                "success": False,
                "error": "Bad Request",
                "message": "O campo 'email' não pode ser vazio."
            }), 400
        update_fields["email"] = email.strip().lower()

    if not update_fields:
        return jsonify({
            "success": False,
            "error": "Bad Request",
            "message": "Ao menos um campo válido ('name' ou 'email') deve ser informado."
        }), 400

    # 4. Aplica a atualização no repositório de persistência
    updated_user = user_repo.update(user_id, update_fields)

    return jsonify({
        "success": True,
        "message": "Usuário atualizado com sucesso.",
        "data": updated_user
    }), 200

def delete_user(user_id):
    """ Controlador para remoção de usuário por ID.
    Método HTTP: DELETE /users/<int:user_id>
    Cenários de retorno:
    - 204 No Content: Usuário localizado e excluído com sucesso (sem corpo de retorno).
    - 404 Not Found: Usuário com o ID especificado não encontrado.
    """
    # 1. Verifica se o recurso existe antes de tentar a exclusão
    existing_user = user_repo.get_by_id(user_id)
    if not existing_user:
        return jsonify({
            "success": False,
            "error": "Not Found",
            "message": f"Não foi possível remover: usuário com ID {user_id} não existe."
        }), 404

    # 2. Executa a exclusão na camada de persistência
    user_repo.delete(user_id)

    # 3. Retorna status HTTP 204 No Content (convenção RESTful rigorosa)
    return Response(status=204)

from flask import Blueprint
from app.controllers.user_controller import update_user, delete_user

# Vinculação das operações ao Blueprint já existente
@app_bp.route('/<int:user_id>', methods=['PUT'])
def put_user(user_id):
    """ Rota RESTful para atualização de registro existente. """
    return update_user(user_id)

@app_bp.route('/<int:user_id>', methods=['DELETE'])
def remove_user(user_id):
    """ Rota RESTful para exclusão de registro. """
    return delete_user(user_id)
```



Código de validação

```
import re # Padrão RFC 5322 simplificado para validação de e-mail EMAIL_REGEX = r'^[\w\.-]+@[\w\.-]+\.\w+$' def validate_user_payload(payload, is_update=False): """ Valida e sanitiza os dados de entrada de usuários. Retorna uma tupla: (erros: list, sanitized_data: dict) """ errors = [] sanitized = {} if not isinstance(payload, dict): return ["O payload da requisição deve ser um objeto JSON válido."], {} name = payload.get("name") email = payload.get("email") # 1. Validação do campo 'name' if not is_update or (is_update and "name" in payload): if name is None or not isinstance(name, str) or not name.strip(): errors.append("O campo 'name' é obrigatório e não pode ser vazio.") elif len(name.strip()) < 3: errors.append("O campo 'name' deve conter ao menos 3 caracteres.") else: sanitized["name"] = name.strip() # 2. Validação do campo 'email' if not is_update or (is_update and "email" in payload): if email is None or not isinstance(email, str) or not email.strip(): errors.append("O campo 'email' é obrigatório e não pode ser vazio.") elif not re.match(EMAIL_REGEX, email.strip()): errors.append("O campo 'email' informado possui formato inválido.") else: sanitized["email"] = email.strip().lower() # 3. Caso seja atualização parcial e nenhum campo válido tenha sido fornecido if is_update and not sanitized and not errors: errors.append("Ao menos um campo válido ('name' ou 'email') deve ser informado para atualização.") return errors, sanitized
```



Evidências de testes

Estudomsb



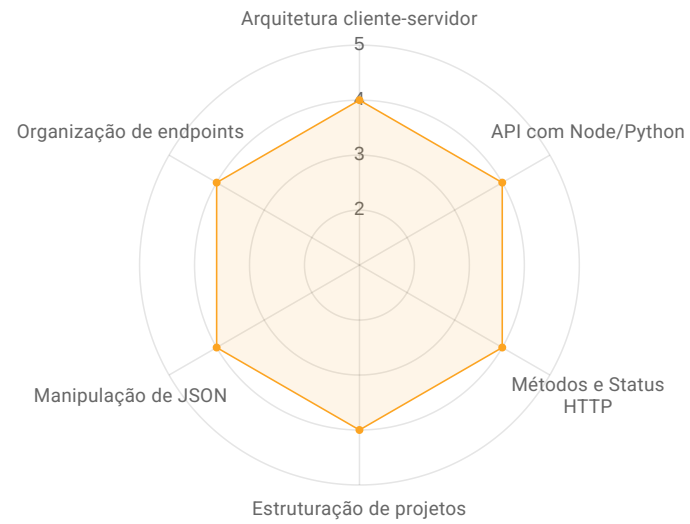
Repositório e README

<https://github.com/Walissoncarval/-https-github.com-walisson-api-connect-walisson-carval-/tree/main> <https://github.com/Walissoncarval/-https-github.com-walisson-api-connect-walisson-carval-.git>



Checklist de qualidade

- Instalei as dependências e configurei um servidor funcional utilizando Node.js/Express ou Python/Flask.
- Estruturei o projeto dividindo as responsabilidades de forma lógica (ex: separação entre rotas, regras e dados).
- Implementei a simulação de persistência de dados através de array em memória ou leitura/escrita de arquivo JSON local.
- Desenvolvi e testei as rotas de listagem geral (GET) e cadastro (POST) operando com o formato JSON.
- Construí as rotas dinâmicas com uso de parâmetros na URL para a busca específica (GET), atualização (PUT/PATCH) e remoção (DELETE).
- Implementei validações de campos obrigatórios, garantindo que usuários não sejam criados sem nome ou e-mail.
- Padronizei as respostas da API e utilizei de forma precisa os códigos de status HTTP apropriados para o sucesso ou falha da operação (200, 201, 204, 400, 404).
- Criei e configurei o repositório no GitHub contendo o código e o arquivo README.md com as orientações de uso e as evidências fotográficas.





Reflexão sobre a aprendizagem

A realização da Experiência Prática II representou um marco decisivo na minha formação técnica, consolidando a transição da análise teórica para a engenharia de software aplicada no desenvolvimento back-end. A construção do MVP da API Connect exigiu ir além de simplesmente fazer o código funcionar, demandando decisões arquiteturais pautadas em manutenibilidade, previsibilidade de comunicação e resiliência de dados. Entre os principais pontos fortes desenvolvidos, destaco o domínio prático da semântica HTTP e do padrão RESTful. A aplicação rigorosa dos métodos (GET, POST, PUT, DELETE) e a calibragem exata dos códigos de status — como a utilização de 201 Created para novas inserções, 204 No Content para exclusões e 404 Not Found para registros inexistentes — garantiram conformidade profissional ao sistema. Outro pilar fundamental foi a adoção do princípio da Separação de Responsabilidades (SoC). Estruturar a aplicação em Python/Flask dividindo rotas (via Blueprints), controladores de negócio, módulo dedicado de validação e repositório de persistência evitou código acoplado e tornou o projeto escalável. Complementarmente, a criação de regras de validação prévia blindou a persistência contra entradas inválidas, enquanto a padronização das respostas em envelopes JSON facilitou a previsibilidade na integração com clientes externos. O fechamento do ciclo com testes manuais cobrindo cenários de sucesso e exceção, associado à documentação técnica no README.md e ao versionamento no GitHub, reforçou uma postura profissional alinhada às exigências do mercado. Como oportunidades de melhoria e próximos passos, identifico a necessidade de evoluir a persistência provisória em JSON para um banco de dados relacional robusto (como PostgreSQL) integrado a um ORM (como SQLAlchemy). Além disso, pretendo avançar na implementação de mecanismos de autenticação e autorização stateless via tokens JWT, no controle de acesso com CORS e no desenvolvimento de baterias de testes automatizados com Pytest para suporte a pipelines de CI/CD. Esta experiência fortaleceu de forma significativa minha autonomia técnica, meu raciocínio sistêmico e minha visão sobre o ecossistema server-side. Compreender a API não apenas como linhas de código, mas como um contrato formal de comunicação e uma camada lógica confiável, fornece uma base sólida para atuar com segurança, ética e qualidade em projetos reais de tecnologia.